

Reviewer Pack — Dyadic Martingale Quadratic Variation Lower-Bounds Sign-Rank...

Entry d825aae8b5d5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-26 21:34:00 UTC

Dyadic Martingale Quadratic Variation Lower-Bounds Sign-Rank Discrepancy

Entry ID: d825aae8b5d5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-26 21:34:00 UTC

1. Conjecture

Field A (mathematical branch): Dyadic martingale theory (Burkholder-Davis-Gundy square-function inequalities on the Haar basis) **Field B** (complexity object): Discrepancy-based randomized communication complexity of ± 1 sign matrices

Statement:

For any ± 1 matrix M of size $2^n \times 2^n$, let $Q(M)$ be the maximum, over all dyadic Haar-tile partitions P of $[0,1]^2$, of the L^2 quadratic-variation sum $\sqrt{\sum_{R \in P} (\text{avg}_R M)^2 \cdot \text{area}(R)}$ restricted to tiles where $|\text{avg}_R M| > 1/2$. We conjecture $\text{disc}(M) \geq c \cdot 2^{-n} \cdot Q(M)$ for an absolute constant $c > 0$, equivalently the randomized public-coin communication complexity satisfies $R_{1/3}(M) \geq \log_2(Q(M)) - O(1)$. One ± 1 matrix on $n \leq 10$ with $\text{disc}(M) < 0.4 \cdot 2^{-n} \cdot Q(M)$ refutes it.

Rationale (proposer's reasoning):

Discrepancy is classically controlled by spectral/cut norms, but Haar-tile quadratic variation captures *localized* sign concentration that cut norms average away; BDG-type square-function inequalities should translate concentrated Haar mass into a discrepancy witness. This bridges martingale square functions (rarely invoked in CC) with Sherstov-style discrepancy bounds and avoids algebrization since the construction is purely combinatorial on the matrix entries.

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2a3b249ce721c953

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all generated ± 1 matrices ($n=3..10$, 4 families, 5 seeds each = 160 instances), the ratio $r(M) = \text{disc}(M) \cdot 2^n / Q(M)$ must satisfy $r(M) \geq 0.4$ for $\geq 95\%$ of instances, with mean $r \geq 0.6$. Any single instance with $r < 0.4$ falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 13 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Haar basis dyadic martingale discrepancy communication complexity sign matrix - Burkholder square function quadratic variation sign-rank lower bound - dyadic tile decomposition discrepancy randomized communication complexity ± 1 matrix

Top relevant hits considered: - [http://arxiv.org/abs/2401.14623v1] Structure in Communication Complexity and Constant-Cost Complexity Classes - [http://arxiv.org/abs/2501.18069v1] Local Basis Transformation to Mitigate Negative Sign Problems - [http://arxiv.org/abs/1101.3039v1] Freedman's inequality for matrix martingales - [http://arxiv.org/abs/2509.08842v2] Explicit lower bounds for opaque sets of unit square and unit disc - [http://arxiv.org/abs/1503.07648v2] Sign rank versus VC dimension - [http://arxiv.org/abs/0906.0693v3] An improved lower bound on the counterfeit coins problem - [http://arxiv.org/abs/1608.02791v2] The Laser calibration of the Atlas Tile Calorimeter during the LHC run 1 - [http://arxiv.org/abs/2410.21906v1] Hartwig-Spindelböck decomposition of dual complex matrix

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import json
from itertools import product

def hadamard(n):
    if n == 1:
        return [[1]]
    H = hadamard(n // 2)
    size = len(H)
    new_H = [[0 for _ in range(size * 2)] for _ in range(size * 2)]
    for i in range(size):
        for j in range(size):
            new_H[i][j] = H[i][j]
            new_H[i][j + size] = H[i][j]
            new_H[i + size][j] = H[i][j]
            new_H[i + size][j + size] = -H[i][j]
    return new_H

def random_bernoulli(n):
```

```

    return [[random.choice([-1, 1]) for _ in range(2**n)] for _ in range(2**n)]

def disjointness(n):
    M = [[0 for _ in range(2**n)] for _ in range(2**n)]
    for i in range(2**n):
        M[i][i] = 1
    return M

def planted_low_rank_plus_noise(n, rank=3, noise_level=0.5):
    U = [[random.choice([-1, 1]) for _ in range(rank)] for _ in range(2**n)]
    V = [[random.choice([-1, 1]) for _ in range(rank)] for _ in range(2**n)]
    M = [[sum(U[i][k] * V[j][k] for k in range(rank)) + random.gauss(0, noise_level) for j in range(2**n)] for i in range(2**n)]
    return M

def avg(M, R):
    sum_val = 0
    area = len(R)
    for r in R:
        for c in R:
            sum_val += M[r][c]
    return sum_val / area

def quadratic_variation(M, P):
    total = 0
    for R in P:
        avg_R = avg(M, R)
        if abs(avg_R) > 1/2:
            total += (avg_R ** 2) * len(R)
    return math.sqrt(total)

def generate_partitions(n):
    partitions = []
    for depth in range(1, n + 1):
        current_partition = [[i for i in range(2**n)]]
        while len(current_partition[0]) > 1:
            new_partition = []
            for R in current_partition:
                mid = len(R) // 2
                new_partition.append(R[:mid])
                new_partition.append(R[mid:])
            current_partition = new_partition
        partitions.extend(current_partition)
    return partitions

def discrepancy(M):
    n = int(math.log2(len(M)))
    if n <= 6:
        rectangles = list(product(range(2**n), repeat=4))
        min_disc = float('inf')
        for r1, c1, r2, c2 in rectangles:
            submatrix = [row[c1:c2+1] for row in M[r1:r2+1]]
            avg_val = sum(sum(row) for row in submatrix) / ((r2 - r1 + 1) * (c2 - c1 + 1))
            disc = abs(avg_val)
            if disc < min_disc:
                min_disc = disc
    return min_disc

```

```

else:
    partitions = generate_partitions(n)
    max_disc = 0
    for partition in partitions:
        avg_val = avg(M, partition)
        if abs(avg_val) > 1/2:
            disc = (avg_val ** 2) * len(partition)
            if disc > max_disc:
                max_disc = disc
    return math.sqrt(max_disc)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [3, 4, 5, 6, 7, 8, 9, 10]
    results = []

    for n in n_values:
        M = random.choice([hadamard(n), random_bernoulli(n), disjointness(n), planted_low_rank_plu
        disc_M = discrepancy(M)
        Q_M = quadratic_variation(M, generate_partitions(n))
        r_M = disc_M * 2**n / Q_M
        results.append({
            "metric_name": "discrepancy_ratio",
            "metric_value": r_M,
            "instances_tested": 1,
            "conjecture_holds": r_M >= 0.4,
            "counterexample": "" if r_M >= 0.4 else f"n={n}, disc(M)={disc_M}, Q(M)={Q_M}"
        })

    return {
        "seed": seed,
        "metric_name": "discrepancy_ratio",
        "metric_value": sum(result["metric_value"] for result in results),
        "instances_tested": len(results),
        "conjecture_holds": all(result["conjecture_holds"] for result in results),
        "counterexample": next((result["counterexample"] for result in results if not result["conjecture_holds"]))
    }

if __name__ == "__main__":
    seeds = list(map(int, sys.argv[1:])) or [11, 23, 37, 53, 71]
    all_results = []

    for seed in seeds:
        result = run_trial(seed)
        print(json.dumps({"TRIAL": result}))
        all_results.append(result)

    mean_r = sum(result["metric_value"] for result in all_results) / len(all_results)
    support_fraction = sum(1 for result in all_results if result["conjecture_holds"]) / len(all_results)

    if support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_r} std=0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in all_results):
        first_failing_seed = next(seed for seed, result in zip(seeds, all_results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{next(result['counterexample'] for result in all_results if not result['conjecture_holds'])}\"")
    else:

```

```
print("RESULT: INCONCLUSIVE insufficient data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_642011e0.py", line 131, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_642011e0.py", line 106, in run_trial
    disc_M = discrepancy(M)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_642011e0.py", line 83, in discrepancy
    avg_val = sum(sum(row) for row in submatrix) / ((r2 - r1 + 1) * (c2 - c1 + 1))
               ^~~~~~
ZeroDivisionError: division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a ZeroDivisionError in the discrepancy() function before any instances were evaluated, so neither the support nor falsification condition can be assessed.
| next: Fix the discrepancy() submatrix indexing bug (guard against empty/degenerate rectangles or use (r2-r1)*(c2-c1) with proper bounds) and rerun the 160-instance battery.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	15455
2	preregistration	claude_max	opus	0	0	4645
3	novelty	claude_max	opus	0	0	4008
4	novelty	claude_max	opus	0	0	9030
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16535
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15837
7	judge	claude_max	opus	0	0	4064

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 69573 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/d825aae8b5d5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d825aae8b5d5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d825aae8b5d5.tar.gz` (if generated)